

Arquitetura e Organização de Computadores

Estruturas de Interconexão do Computador

Rodrigo Calvo
rcalvo@uem.br

Departamento de Informática – DIN
Universidade Estadual de Maringá – UEM

O Computador IAS

- Foi o primeiro computador eletrônico, construído pelo Instituto de Estudos Avançados de Princeton (IAS).
 - O artigo descrevendo o projeto foi editado por John von Neumann em 1952.
- Máquina binária com uma palavra de 40 bits
 - Armazenava duas instruções de 20 bits em cada palavra.

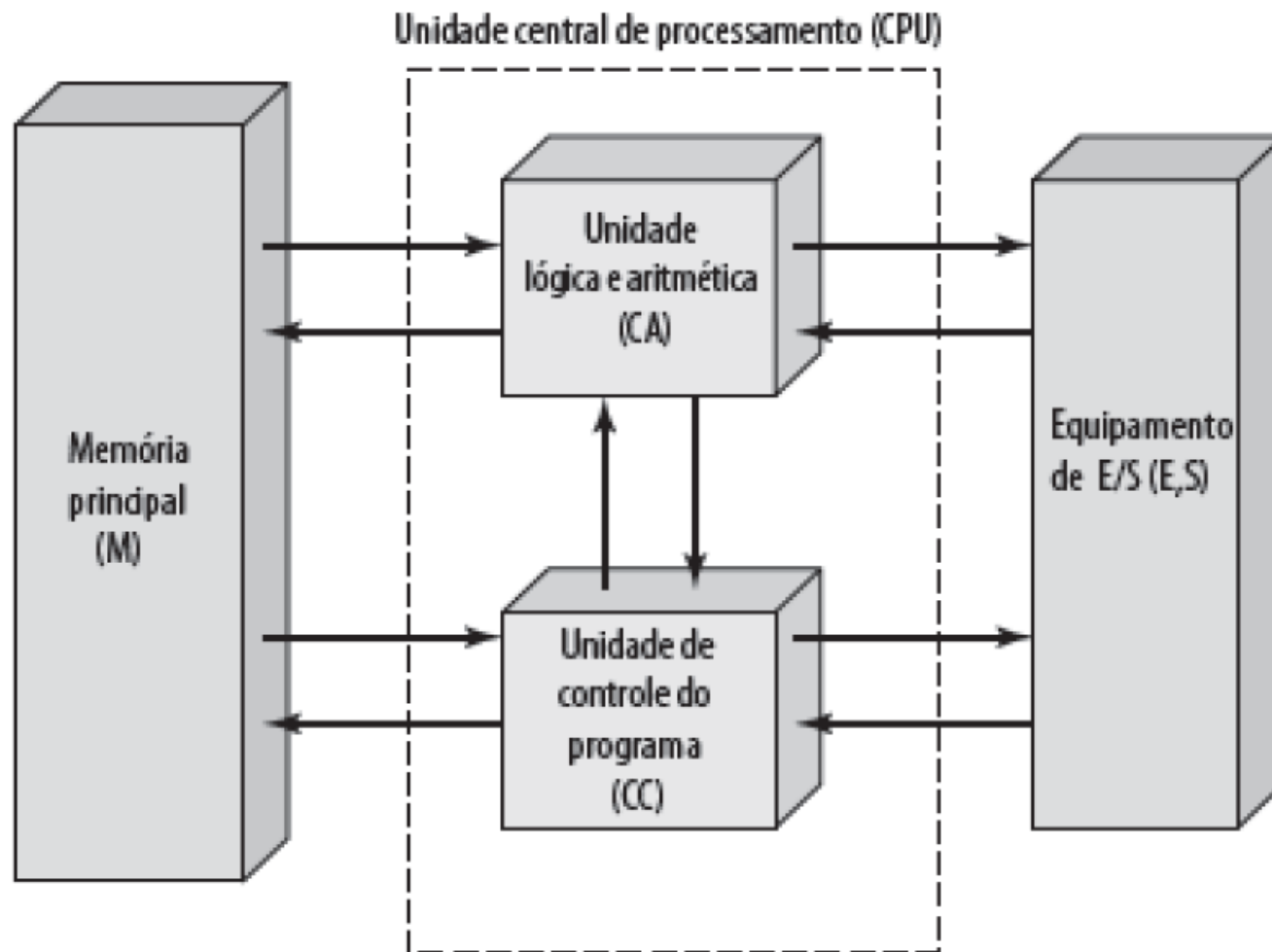
O Computador IAS

- Motivação:
 - Reduzir o tempo necessário para a execução de uma instrução no ENIAC
 - Armazenamento de instruções e dados em uma mesma memória
- Conceito de programa armazenado
- John von Neumann e colegas iniciaram o projeto de um novo computador de programa armazenado, conhecido como IAS

O Computador IAS

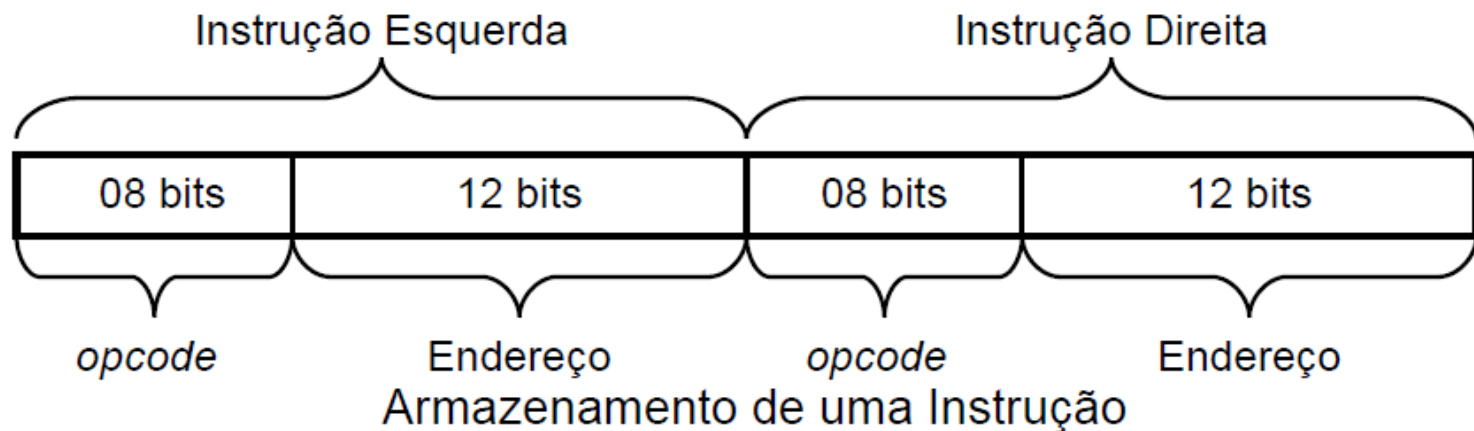
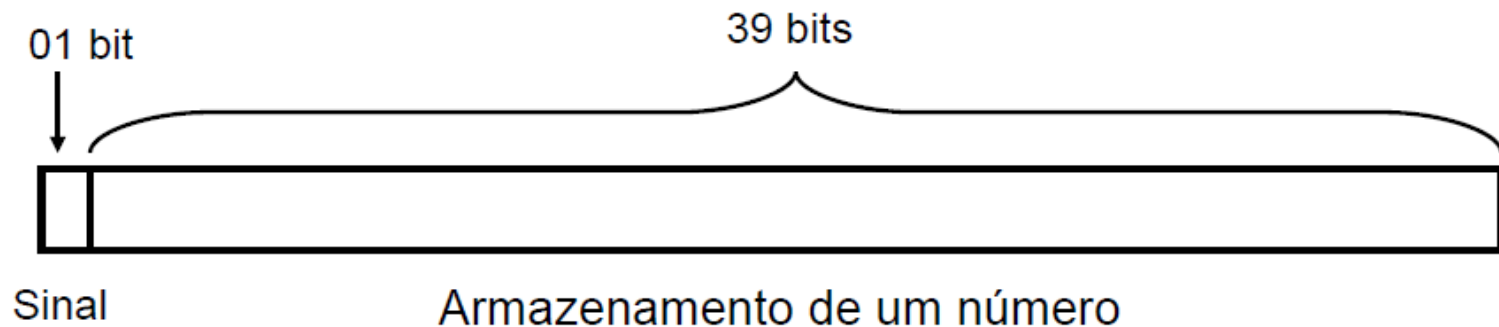
- A memória tinha capacidade para 1024 palavras (5,1 KBytes).
- Tinha dois registradores de “uso geral”:
 - Acumulador (AC); e
 - Multiplicador/Quociente (MQ).
- O computador IAS foi o primeiro projeto a misturar programas e dados numa única memória.

O Computador IAS

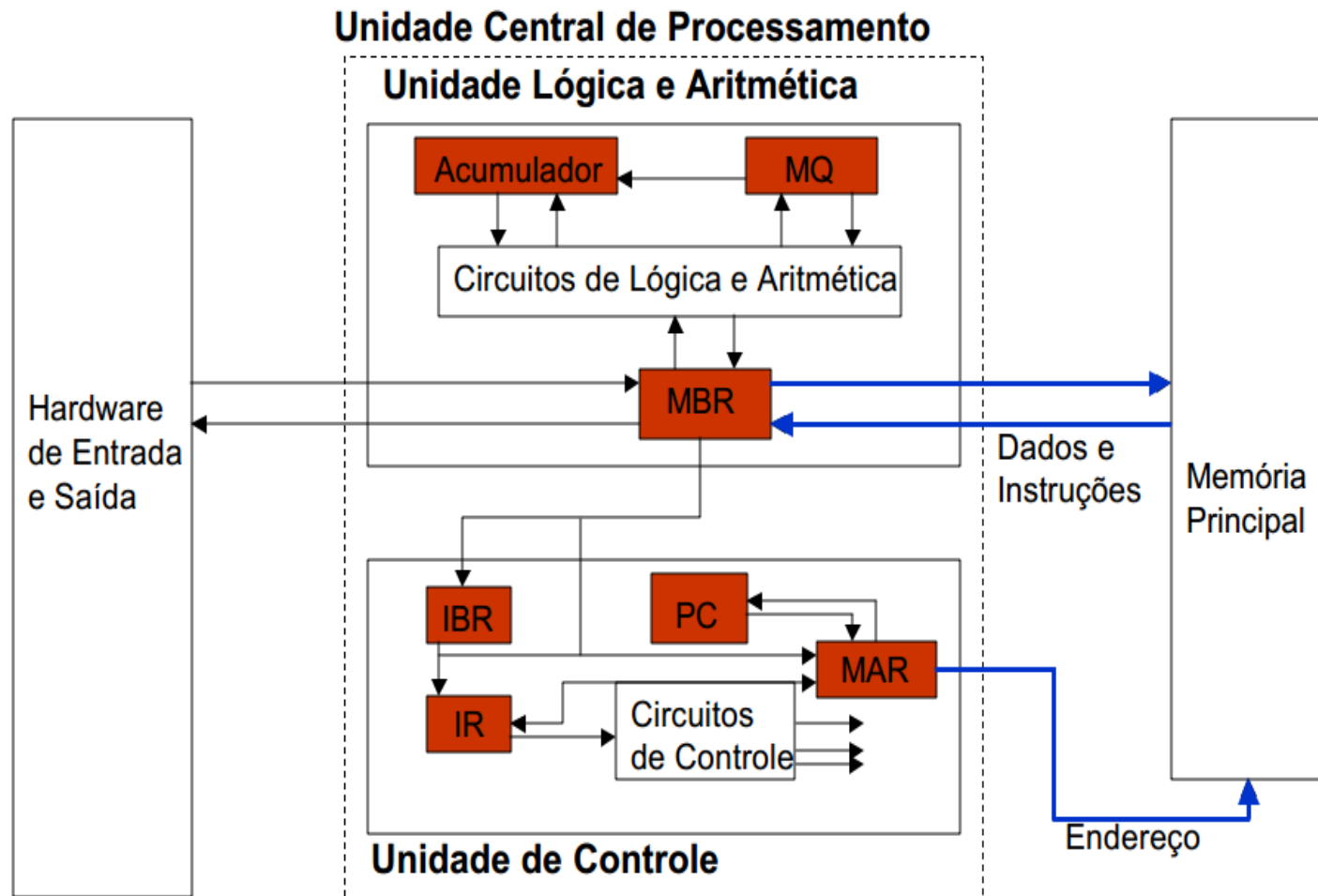


O Computador IAS

- A Memória Principal do IAS possui palavras de 40 bits
 - dados ou instruções de máquina são armazenados por palavra



Registradores do IAS



Registradores do IAS

Registrador	Size (bits)	Função
Contador de Programa (PC)	12	Armazena endereço da próxima instrução
Acumulador (AC)	40	Armazenamento temporário de dados
Multiplier quotient (MQ)	40	Armazenamento temporário de dados
Buffer de dados/instrução (MBR)	40	Dados de leitura/escrita de memória
Buffer de instrução (IBR)	20	Armazena instrução direita (bits 20-39)
Register de instrução (IR)	8	Armazena <i>opcode</i> de uma instrução
Endereço de memória (MAR)	12	Armazena endereço de memória de uma instrução

Instruções do IAS

- Instruções de Transferência de Dados

opcode	mnemônico	significado
00001010	LOAD MQ	$AC \leftarrow MQ$
00001001	LOAD MQ,M(X)	$MQ \leftarrow \text{mem}(X)$
00100001	STOR M(X)	$\text{mem}(X) \leftarrow AC$
00000001	LOAD M(X)	$AC \leftarrow \text{mem}(X)$
00000010	LOAD $-M(X)$	$AC \leftarrow -\text{mem}(X)$
00000011	LOAD $ M(X) $	$AC \leftarrow \text{abs}(\text{mem}(X))$
00000100	LOAD $- M(X) $	$AC \leftarrow -\text{abs}(\text{mem}(X))$

Instruções do IAS

- Instruções de Desvio Incondicional

opcode	mnemônico	significado
00001101	JUMP M(X,0:19)	próx. instr.: metade esq. de mem(X)
00001110	JUMP M(X,20:39)	próx. instr.: metade dir. de mem(X)

Instruções do IAS

- Instruções de Desvio Condicional

opcode	mnemônico	significado
00001111	JUMP +M(X,0:19)	se $AC \geq 0$, próx. instr.: metade esq. de mem(X)
00010000	JUMP +M(X,20:39)	se $AC \geq 0$, próx. instr.: metade dir. de mem(X)

Instruções do IAS

- Instruções Aritméticas

opcode	mnemônico	significado
00000101	ADD M(X)	$AC \leftarrow AC + \text{mem}(X)$
00000111	ADD M(X)	$AC \leftarrow AC + \text{mem}(X) $
00000110	SUB M(X)	$AC \leftarrow AC - \text{mem}(X)$
00001000	SUB M(X)	$AC \leftarrow AC - \text{mem}(X) $
00001011	MUL M(X)	$MQ \leftarrow MQ * \text{mem}(X)$
00001100	DIV M(X)	$AC \leftarrow AC / \text{mem}(X)$
00010100	LSH	$AC \leftarrow AC * 2$ (shift esq.)
00010101	RSH	$AC \leftarrow AC / 2$ (shift dir.)

Instruções do IAS

- Instruções de Alteração de Endereço

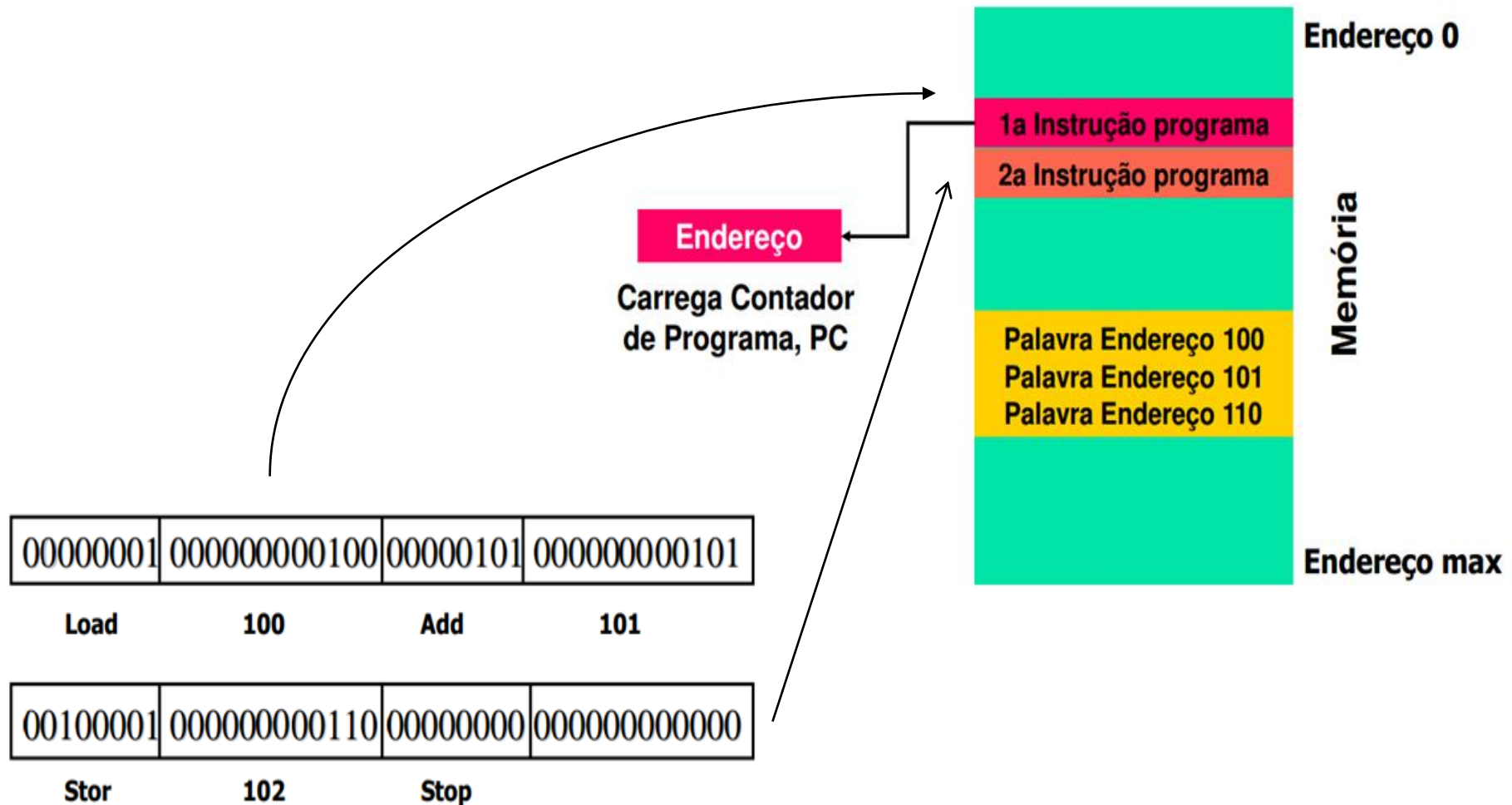
opcode	mnemônico	significado
00010010	STOR M(X,8:19)	substitui o campo de endereço à esq. de mem(X) pelos 12 bits mais à dir. de AC
00010011	STOR M(X,28:39)	subst. o campo de endereço à dir. de mem(X) pelos 12 bits mais à dir. de AC

IAS – Exemplo 1

- Supondo os números armazenados em memória nas posições (endereços) 100 e 101.
- O resultado da soma deve ser armazenado em memória na posição 110.

<i>Instrução</i>	<i>Opcode</i>	<i>Descrição</i>
LOAD M(100)	00000001	AC $\leftarrow$ M(100)
ADD M(101)	00000101	AC $\leftarrow$ AC + M(101)
STOR M(110)	00100001	M(110) $\leftarrow$ AC

IAS – Exemplo 1



IAS – Exemplo 2

- Realização da operação: $(235h * 3h) + (899h * 23h)$
- Primeiro: Como estão os dados na memória?

Endereço	Dado	
009F	00 00 00 00 00	//Armazenamento temporário
0100	00 00 00 02 35	// 0x235
0101	00 00 00 00 03	// 0x3
0102	00 00 00 08 99	// 0x899
0103	00 00 00 00 23	// 0x23

IAS – Exemplo 2

- Realização da operação: $(235h * 3h) + (899h * 23h)$

LOAD MQ, M(0x100)	// Carregar o valor 0x235 em MQ
MUL M(0x101)	// e multiplicar por 0x3
LOAD MQ	// Mover o resultado para AC e
STOR M(0x09F)	// salvar no temporário (0x09F)
LOAD MQ, M(0x102)	// Carregar o valor 0x899 em MQ
MUL M(0x103)	// e multiplicar por 0x23
LOAD MQ	// Mover o resultado para AC e
ADD M(0x09F)	// somar com o temporário.

IAS – Exemplo 3

- Computar o fatorial de N:

```
N = 10;
```

```
fat = 1;
```

```
i = 1;
```

```
faça
```

```
    fat = fat * i;
```

```
    i = i + 1;
```

```
enquanto i <= N;
```

Como estão os dados na memória?

Endereço	Dado
0100	00 00 00 00 0A // N: (N=10)
0101	00 00 00 00 01 // Constante 1
0102	00 00 00 00 01 // fat
0103	00 00 00 00 01 // i

IAS – Exemplo 3

Comeco:

LOAD MQ, M(0x102)	// Carrega fat em MQ
MUL M(0x103)	// Multiplica MQ por i
LOAD MQ	// Carrega resultado em AC
STOR M(0x102)	// Salva resultado em fat
LOAD M(0x103)	// Carrega i em AC
ADD M(0x101)	// Incrementa AC
STOR M(0x103)	// Volta resultado em i novamente
LOAD M(0x100)	// Carrega N em AC
SUB M(0x103)	// $AC = AC - i$
JUMP+ M(Comeco)	// Salta para Comeco se $N - i \geq 0$

IAS – Exercício

- O que o seguinte código faz?

Os dados na memória

Endereço	Dado
0100	00 00 00 00 04 // b
0101	00 00 00 00 01 // res
0102	00 00 00 00 03 // pot
0103	00 00 00 00 01 // i
0104	00 00 00 00 01 // Constante 1

IAS – Exercício

Comeco:

```
LOAD MQ, M(0x101)
MUL  M(0x100)
LOAD MQ
STOR M(0x101)
LOAD M(0x103)
ADD  M(0x104)
STOR M(0x103)
LOAD M(0x102)
SUB  M(0x103)
JUMP+ M(Comeco)
```

```
// Carrega res em MQ
// Multiplica MQ por b
// Carrega resultado em AC
// Salva resultado em res
// Carrega i em AC
// Incrementa AC
// Volta resultado em i novamente
// Carrega pot em AC
// AC = AC - i
// Salta para Comeco se N - i >= 0
```

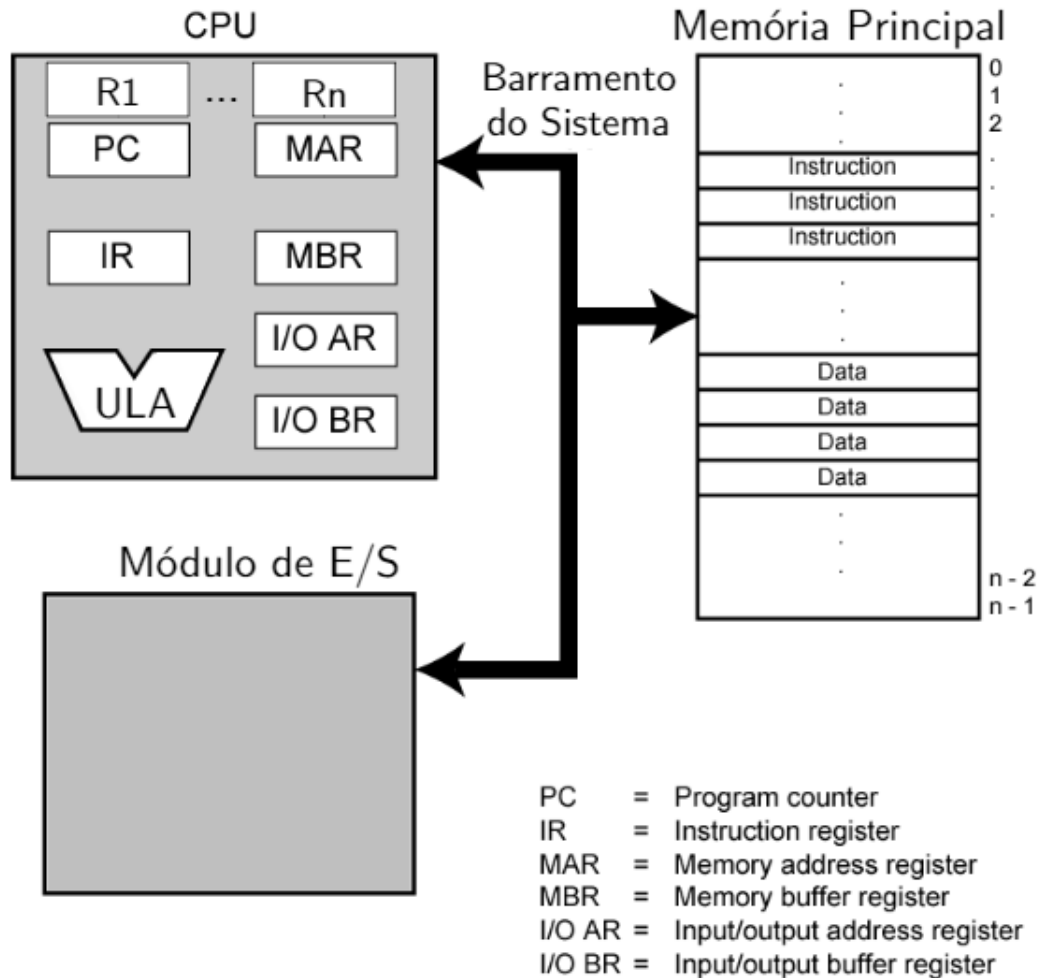
Arquitetura de von Neumann

- Arquitetura de von Neumann é baseada em 3 conceitos:
 - 1) Dados e instruções são armazenados em uma mesma memória;
 - 2) O conteúdo da memória é endereçável sem considerar o tipo de dado;
 - 3) A execução ocorre sequencialmente de uma instrução para outra (exceto que ocorra uma modificação explícita).

Programação em *hardware*

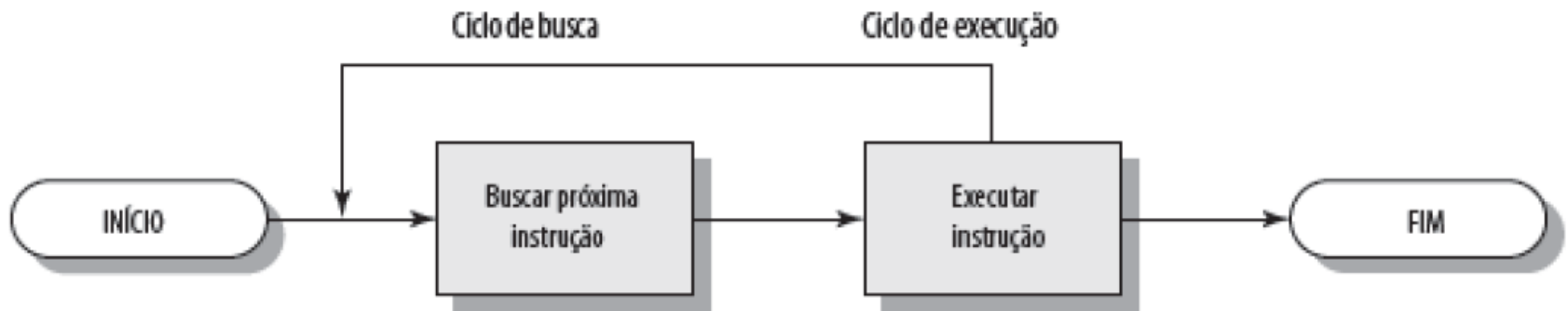
- O armazenamento e operação de dados binários e lógicos são realizados a partir da combinação de componentes lógicos
- A combinação específica de componentes lógicos pode ser feita para realizar cálculos específicos
- Conexão de componentes lógicos → programa *hardwired*
- A combinação de componentes lógicos pode ser feita para a execução de várias funções (uso geral)
 - Sinais de controle são aplicados ao *hardware*

Visão de alto nível



Ciclo de instrução

- Um ciclo de instrução é composto por duas etapas:
 - Ciclo de busca;
 - Ciclo de execução.



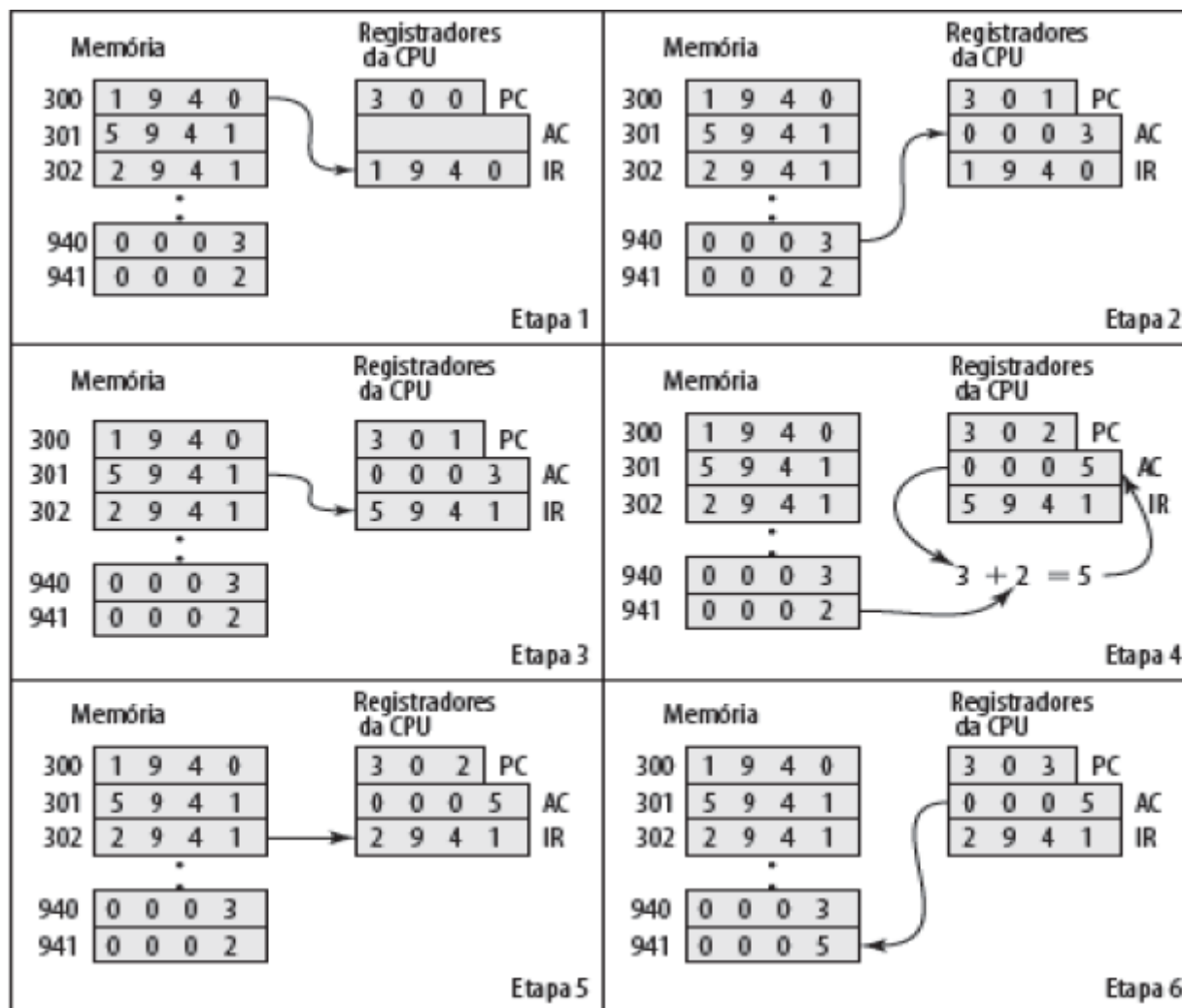
Ciclo de instrução: ciclo de busca

- **Contador de Programa (PC)** mantém endereço da próxima instrução a buscar;
- Processador busca a instrução do local de memória apontado pelo PC;
- Incrementar PC;
 - A menos que seja informado de outra forma.
- Instrução carregada no **Registrador de Instrução (IR)**.

Ciclo de instrução: ciclo de execução

- **Processador** interpreta instrução e realiza as ações exigidas;
- Categoria das ações:
 - Processador – memória;
 - Processador – E/S.
 - Processamento de dados;
 - Controle.

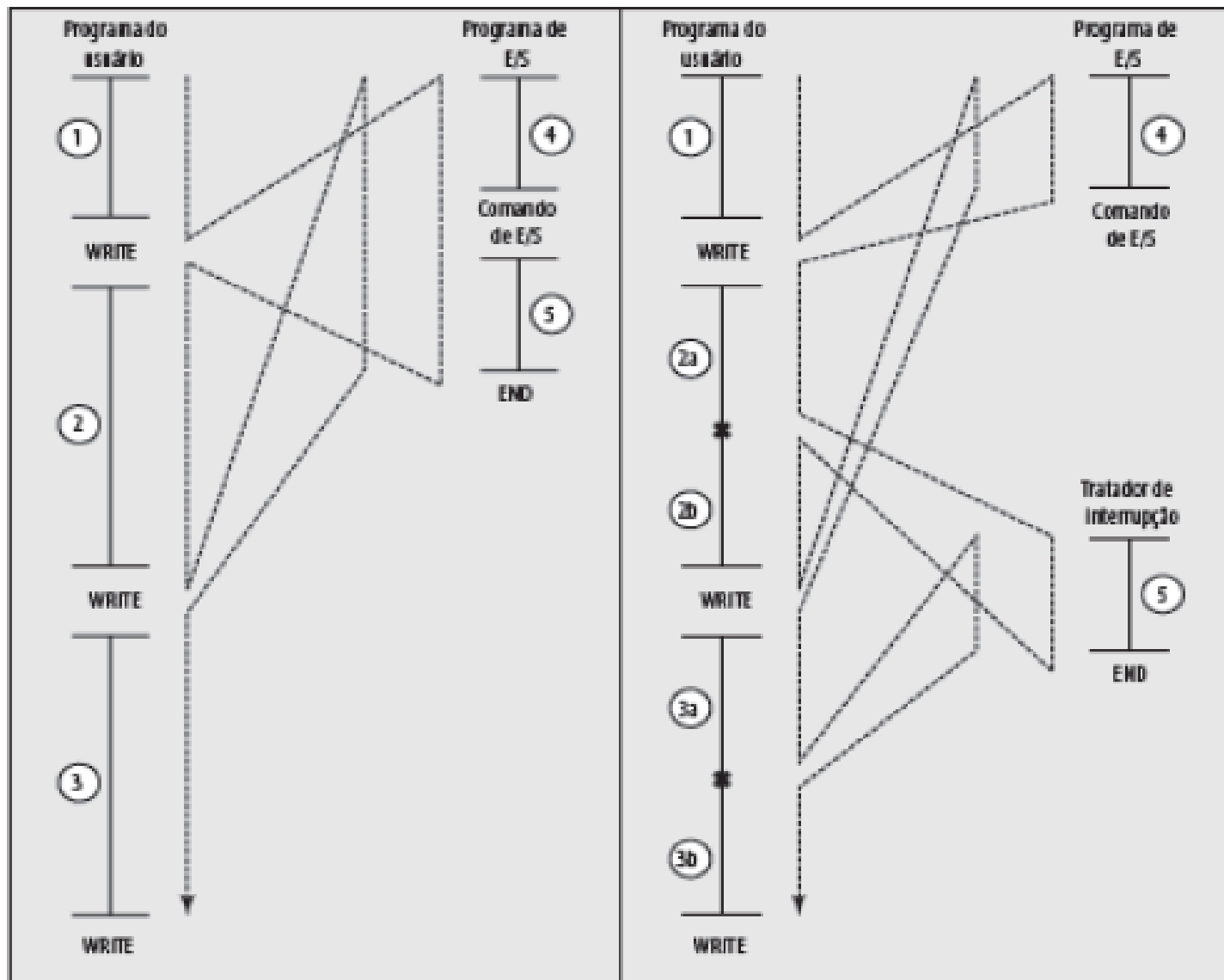
Ciclo de instrução: exemplo



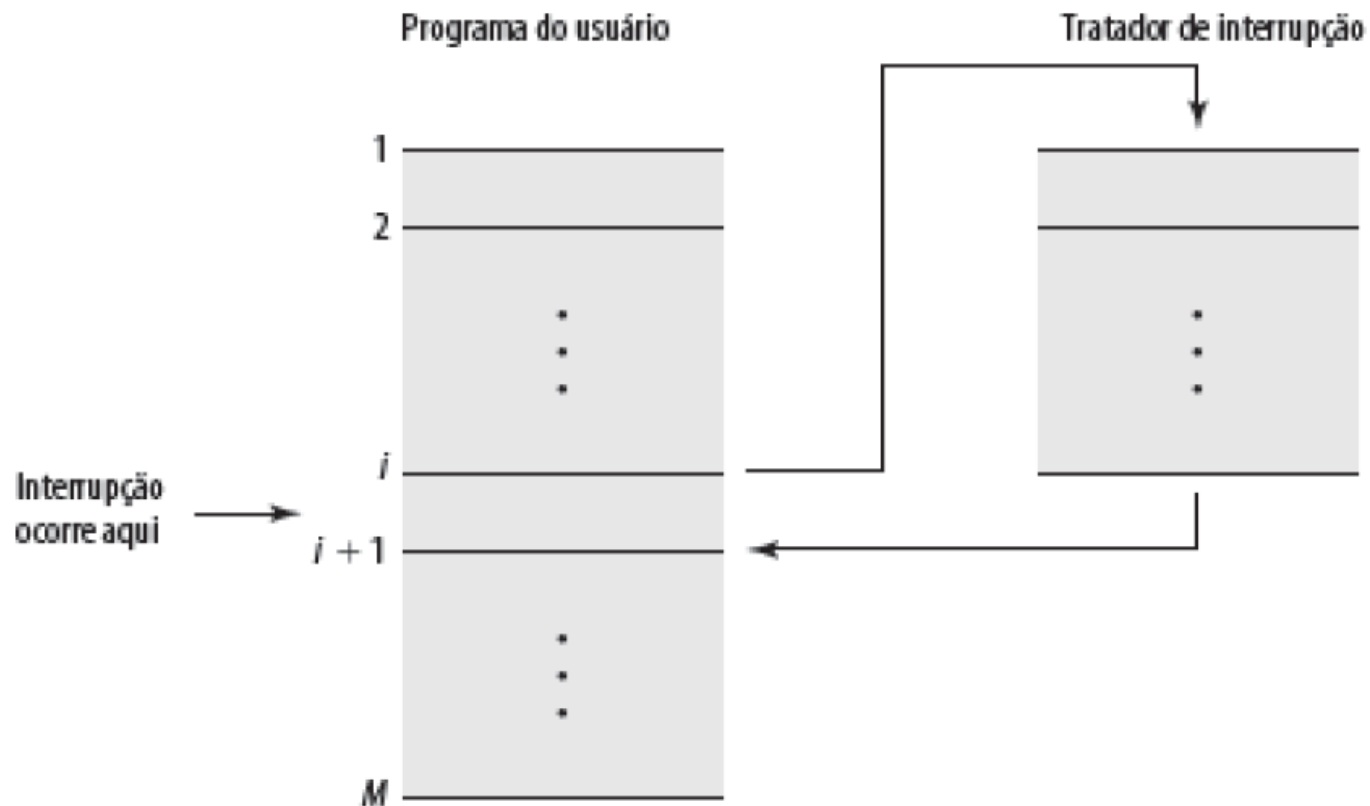
Interrupção

- Mecanismo pelo qual módulos (como E/S) podem interromper a sequência de processamento normal:
- Interrupções podem ser ocasionadas por:
 - Falha de algum **programa**: divisão por zero, acesso à espaço de memória inválido, *overflow*;
 - **Timer**: sinal para indicar o instante da execução de uma função específica;
 - **E/S**: sinalizar o término de uma operação ou ocorrência de erros;
 - **Falha de hardware**: falta de energia.

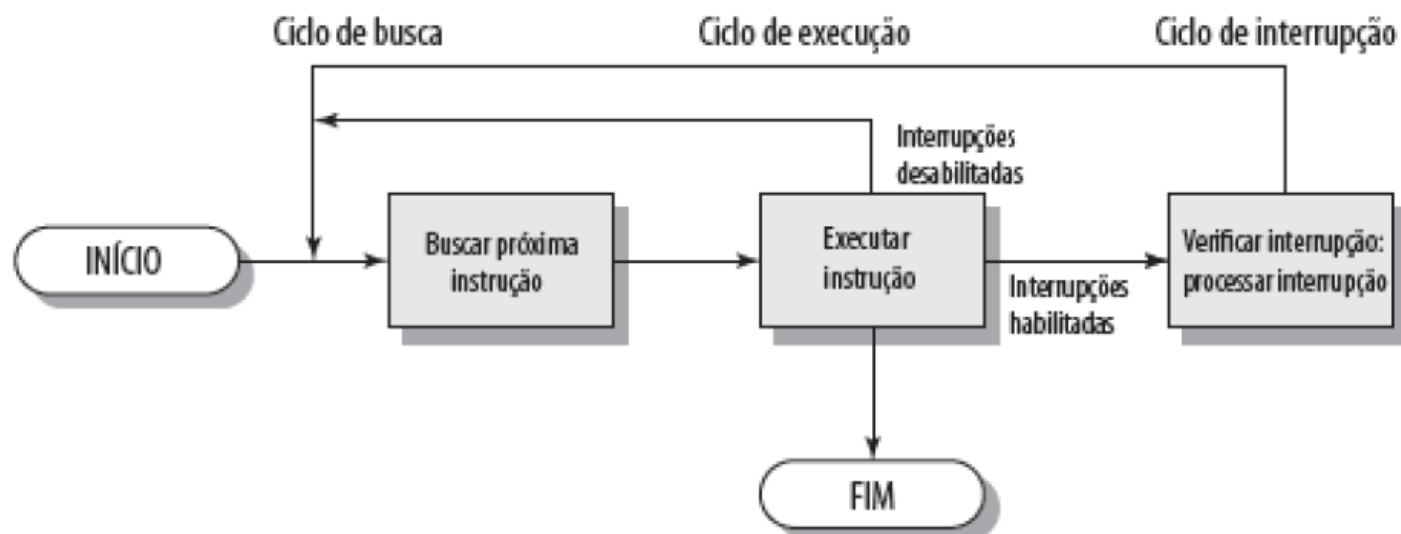
Controle de fluxo de programa



Transferência de controle via interrupções



Ciclo de instrução com interrupções



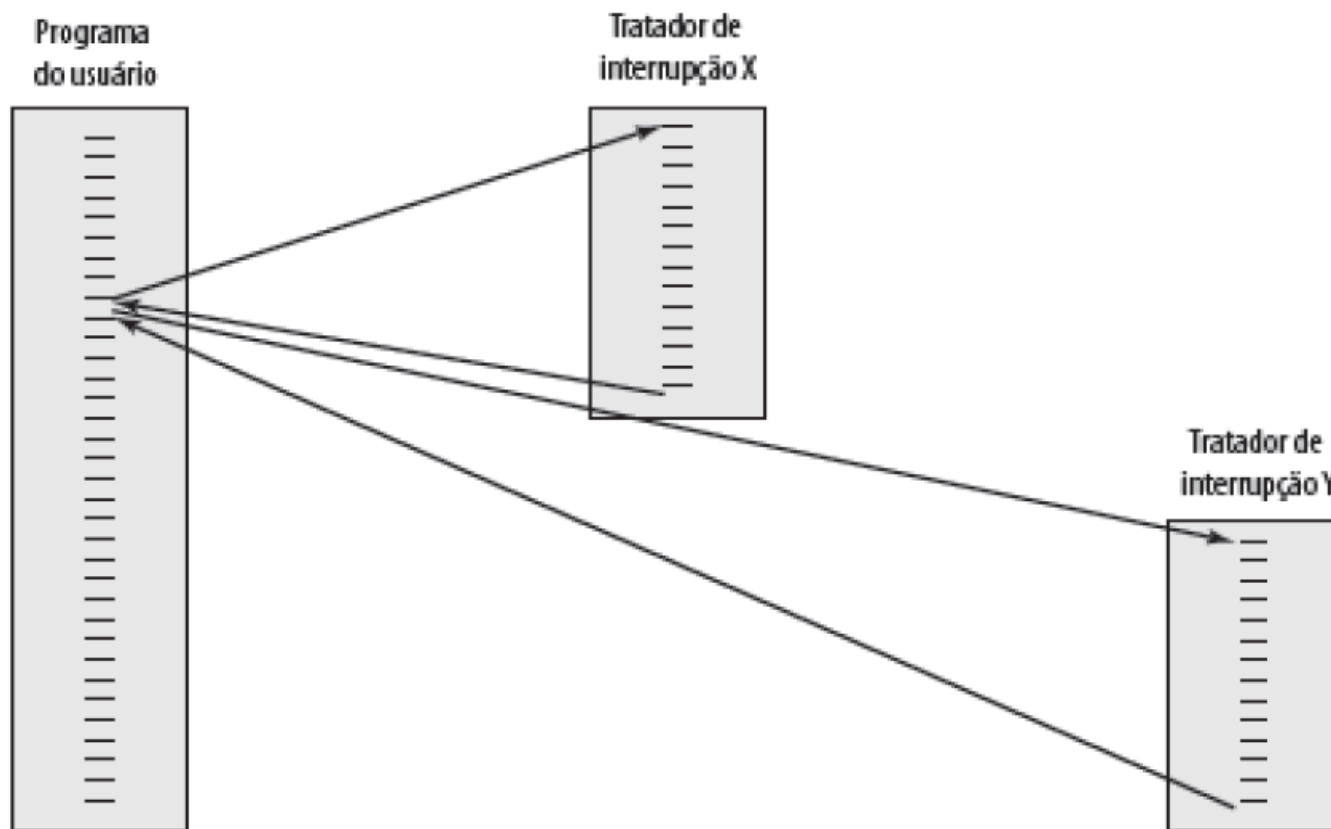
Ciclo de interrupção

- Adicionado ao ciclo de instrução.
- Processador verifica interrupção.
 - Indicado por um sinal de interrupção.
- Se não houver interrupção, busca próxima instrução.
- Se houver interrupção pendente:
 - Suspende execução do programa atual.
 - Salva contexto.
 - Define PC para endereço inicial da rotina de tratamento de interrupção.
 - Interrupção de processo.
 - Restaura contexto e continua programa interrompido.

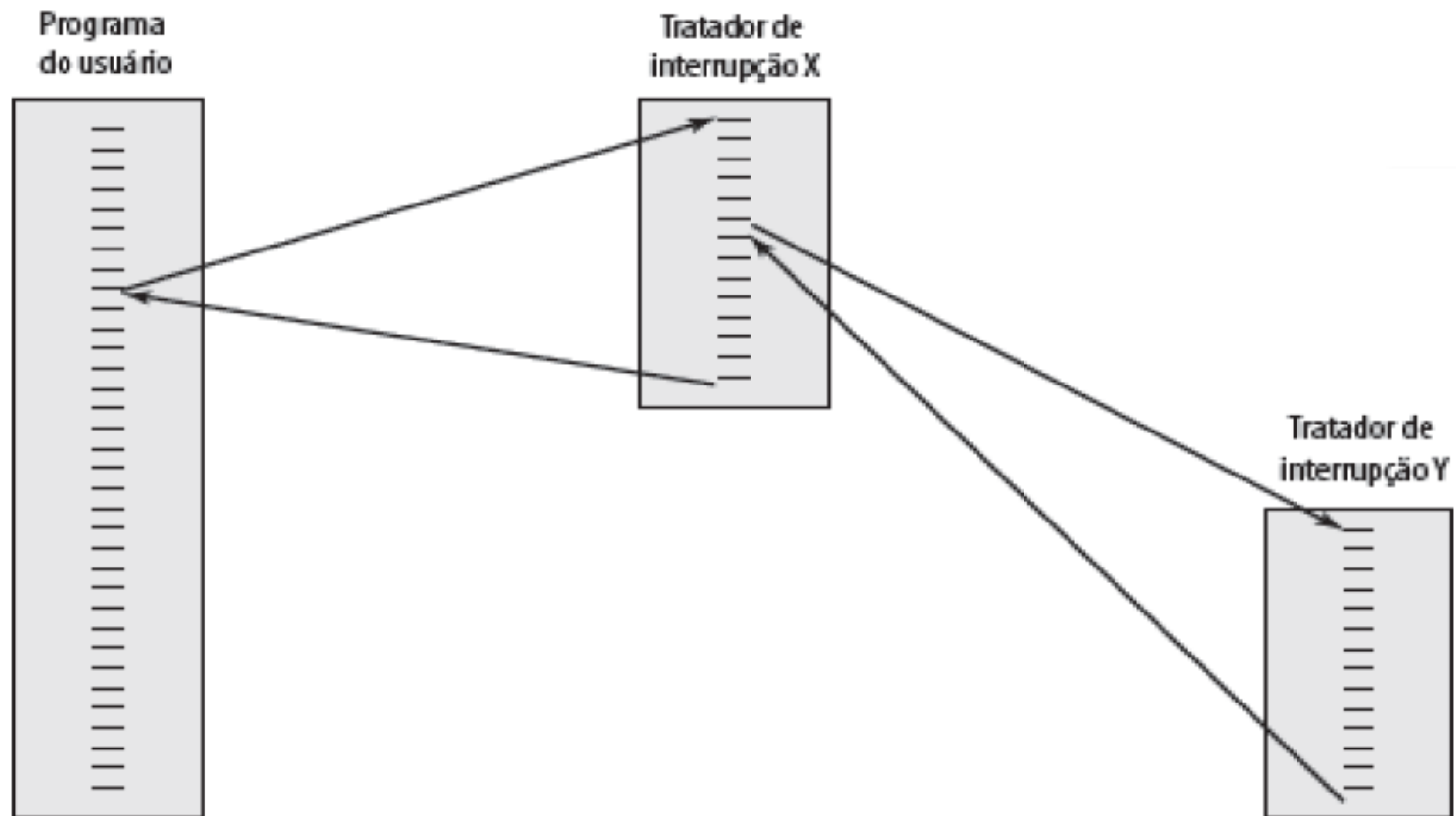
Múltiplas interrupções

- Desativar interrupções:
 - Processador ignorará outras interrupções enquanto processa uma interrupção.
 - Interrupções permanecem pendentes e são verificadas após primeira interrupção ter sido processada.
 - Interrupções tratadas em sequência enquanto ocorrem.
- Definir prioridades:
 - Interrupções de baixa prioridade podem ser interrompidas por interrupções de prioridade mais alta.
 - Quando interrupção de maior prioridade tiver sido processada, processador retorna à interrupção anterior.

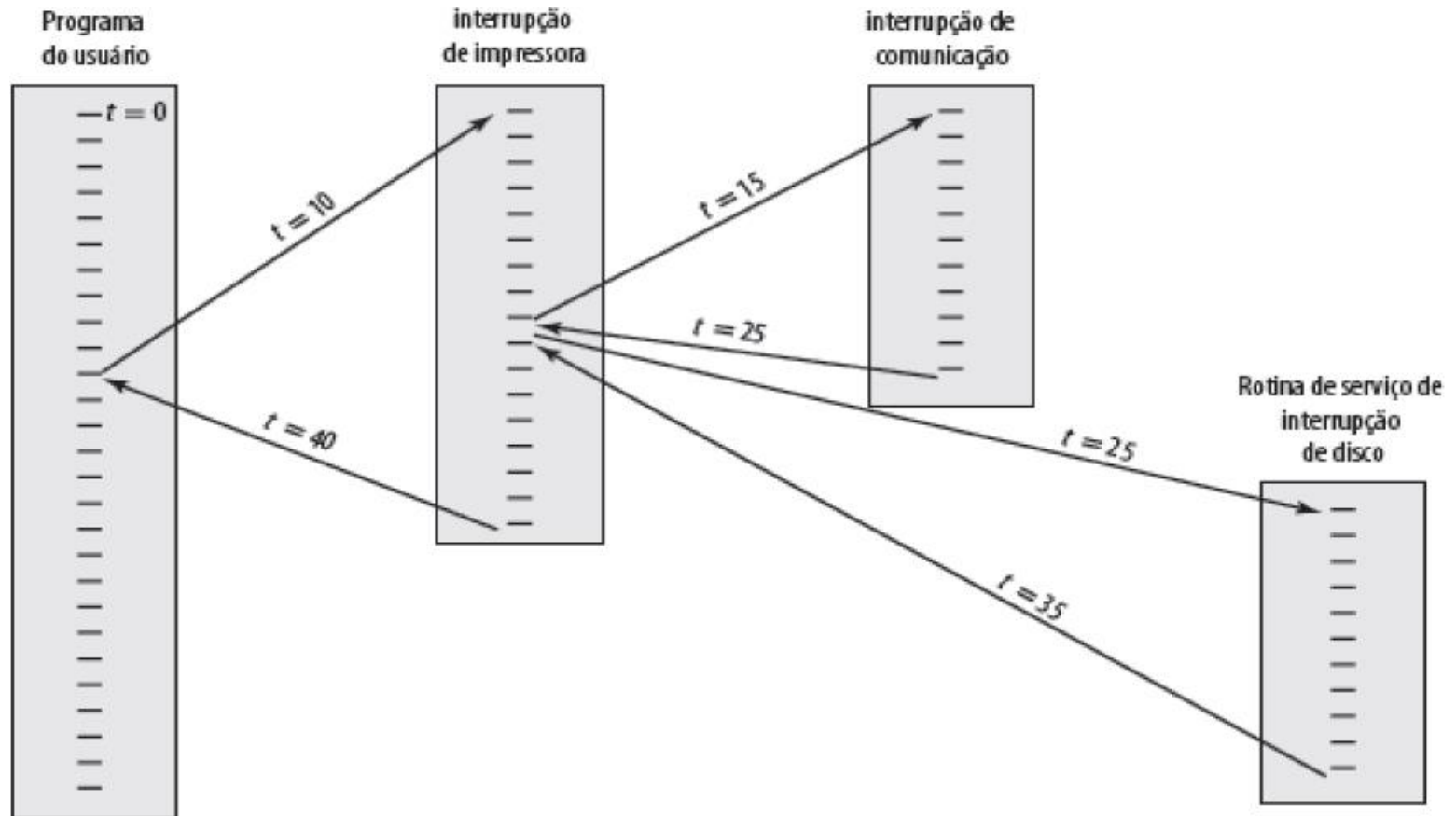
Múltiplas interrupções: sequenciais



Múltiplas interrupções: aninhadas



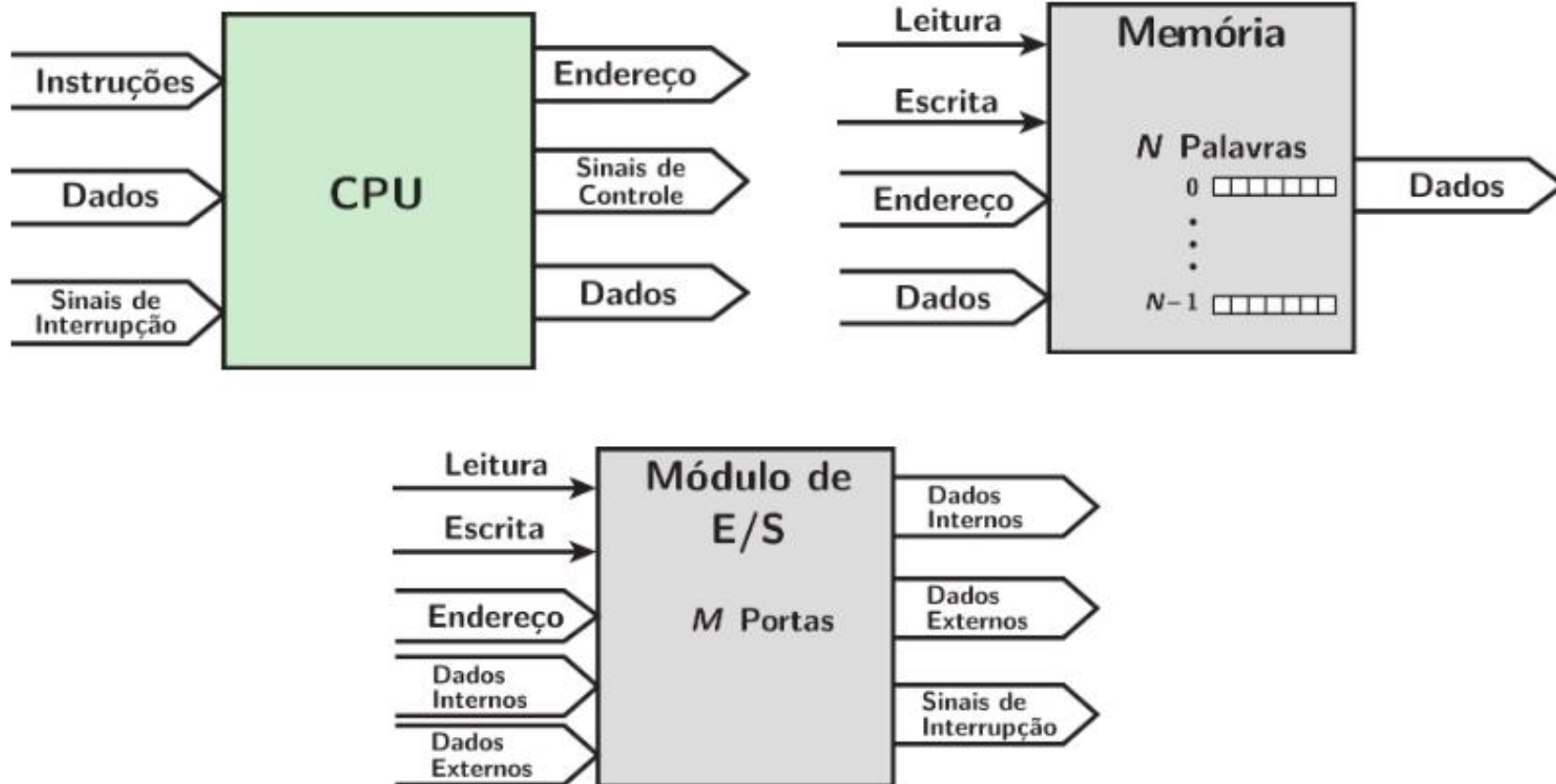
Múltiplas interrupções: prioridade



Conexões

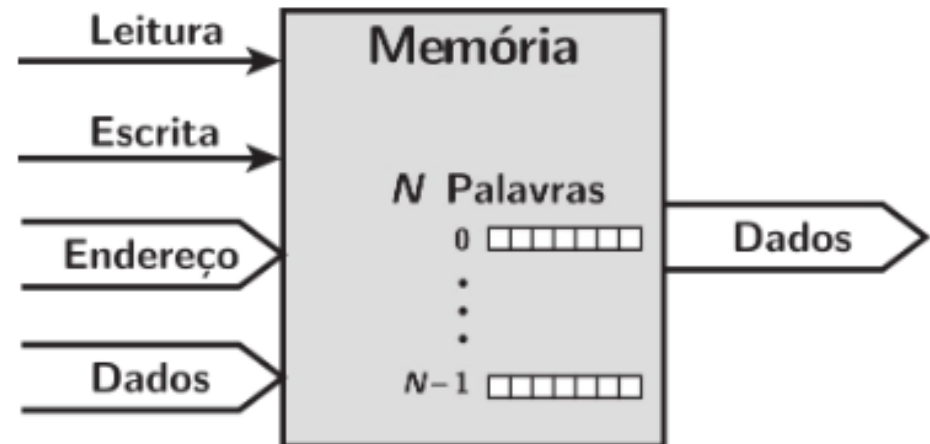
- Todas as unidades de um sistema computacional precisam estar conectadas.
- Tipo diferente de conexão para tipo diferente de módulo:
 - Memória;
 - Módulo de Entrada/Saída;
 - CPU.

Módulos de um Computador



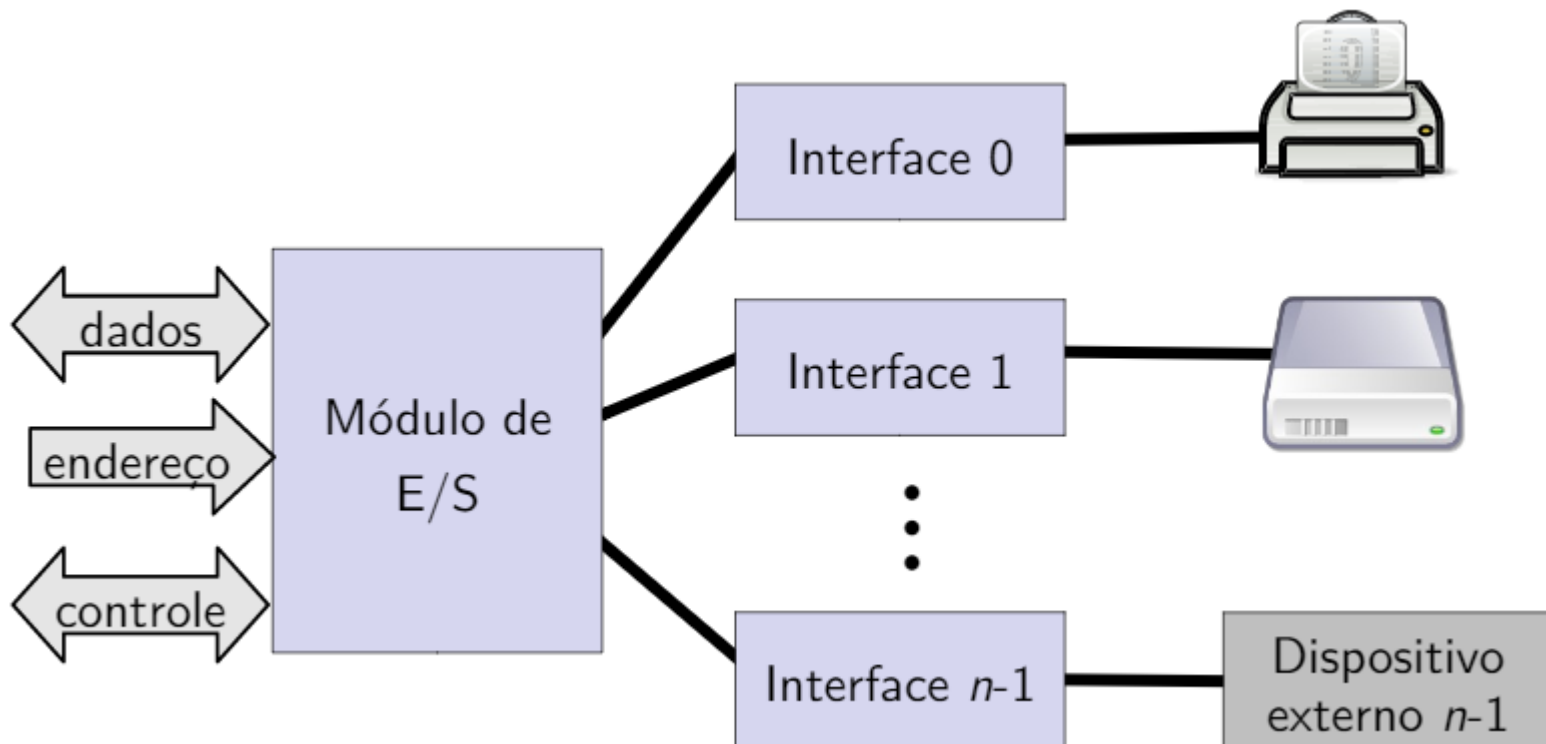
Conexão da Memória Principal

- Tipicamente, a memória principal é composta de N palavras de mesmo tamanho.
- Recebe e envia palavras:
 - Endereços (de posições);
 - Dados.
- Recebe sinais de controle:
 - Leitura;
 - Escrita.



Subsistema de Entrada/Saída

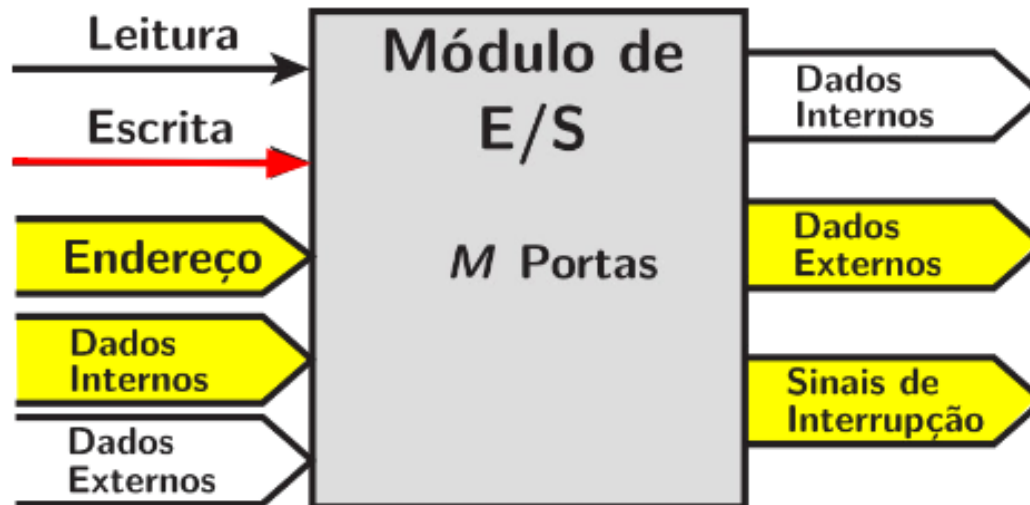
- Similar à memória, do ponto de vista da CPU;



- Cada interface (porta) é identificada por um endereço.

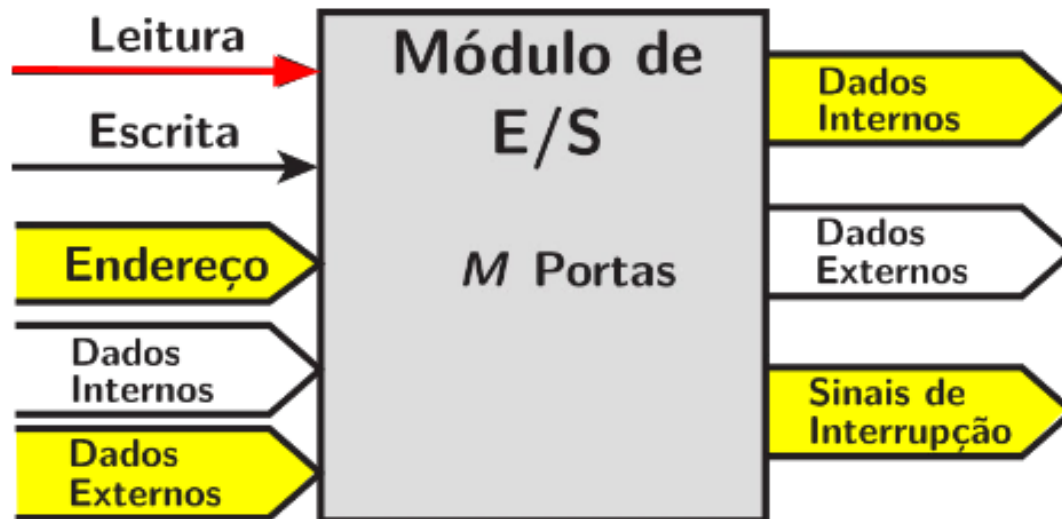
Conexão de Entrada/Saída

- Operação de Saída
 - Recebe dados do computador;
 - Envia dados para a interface de um dispositivo externo.



Conexão de Entrada/Saída

- Operação de Entrada
 - Recebe dados da interface de um dispositivo externo;
 - Envia dados para o computador.

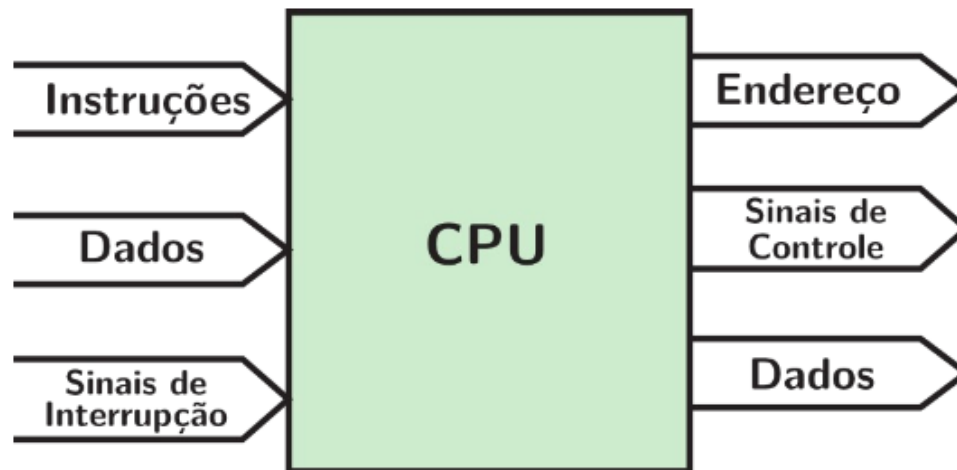


Conexão de Entrada/Saída

- Recebe sinais de controle do computador.
- Envia sinais de controle para os dispositivos externos (periféricos).
 - Ex.: Ligar motor do disco rígido (HD);
- Recebe endereços do computador.
 - Ex.: Número da porta para identificar periférico;
- Envia sinais de interrupção (controle).

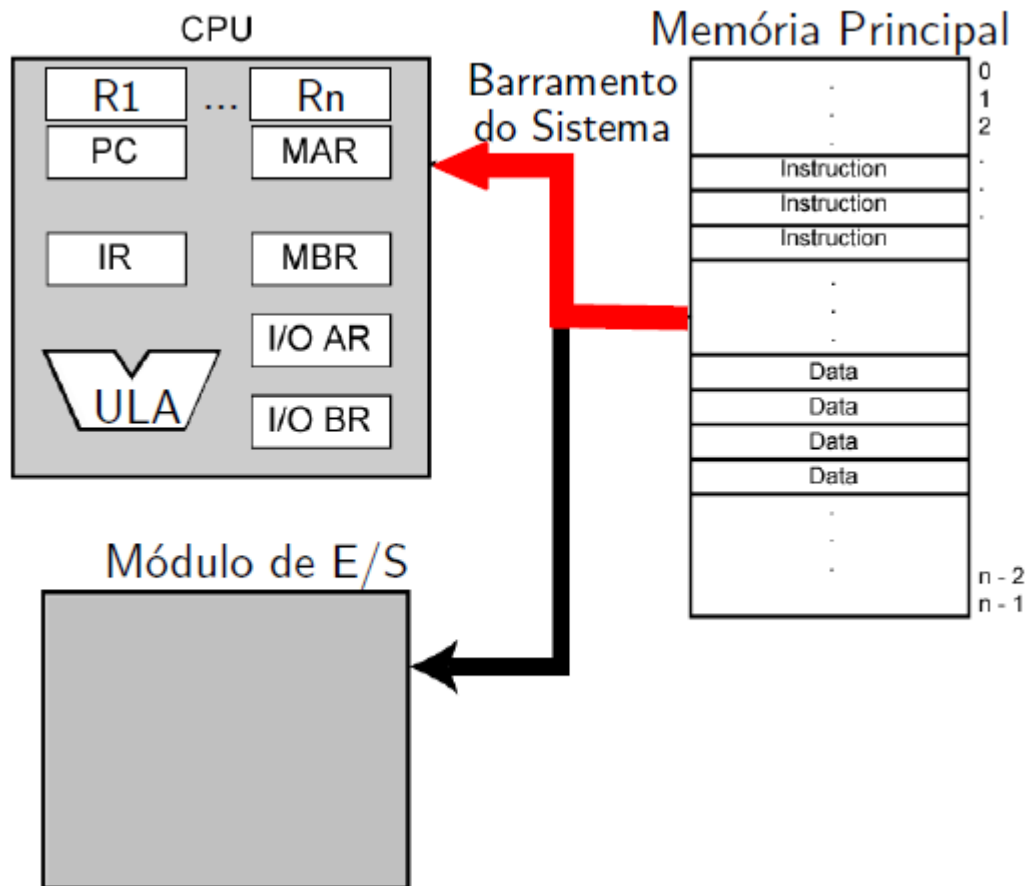
Conexão da CPU

- Lê instruções e dados.
- Escreve dados (após processamento).
- Envia sinais de endereçamento para outras unidades.
- Envia sinais de controle para outras unidades.
- Recebe (e processa) as interrupções.



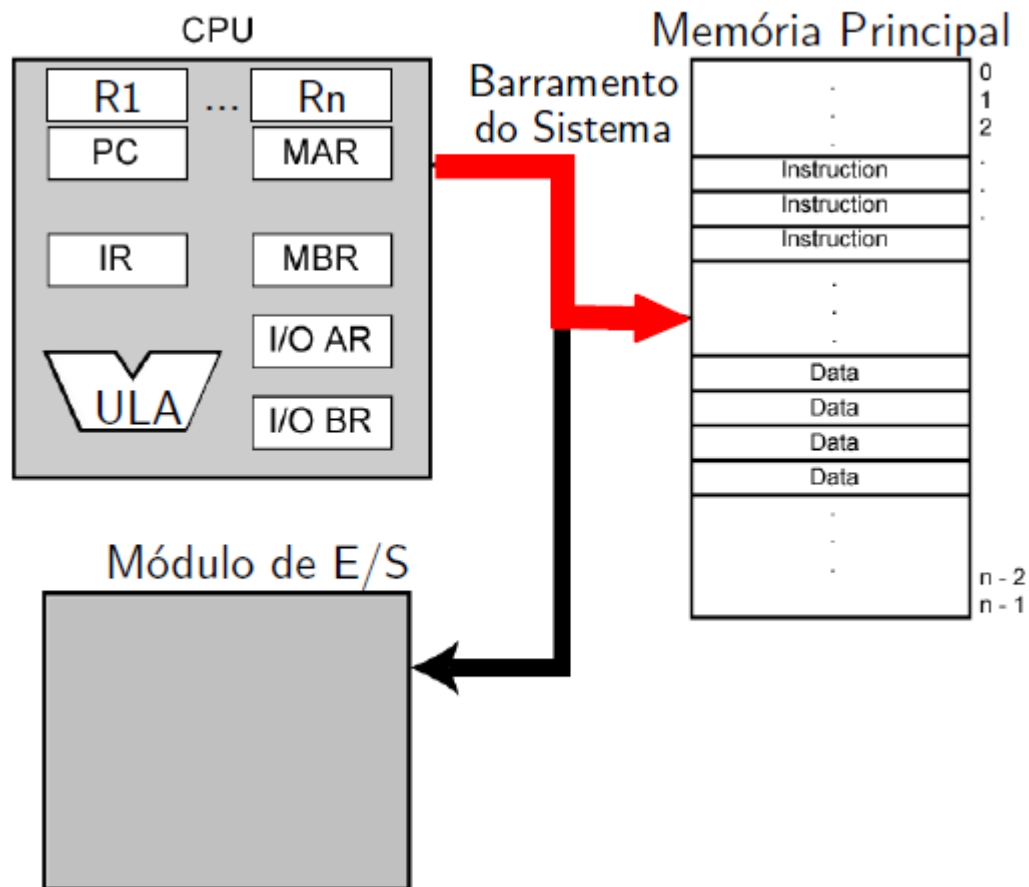
Tipos de Transferência

- Memória principal para processador



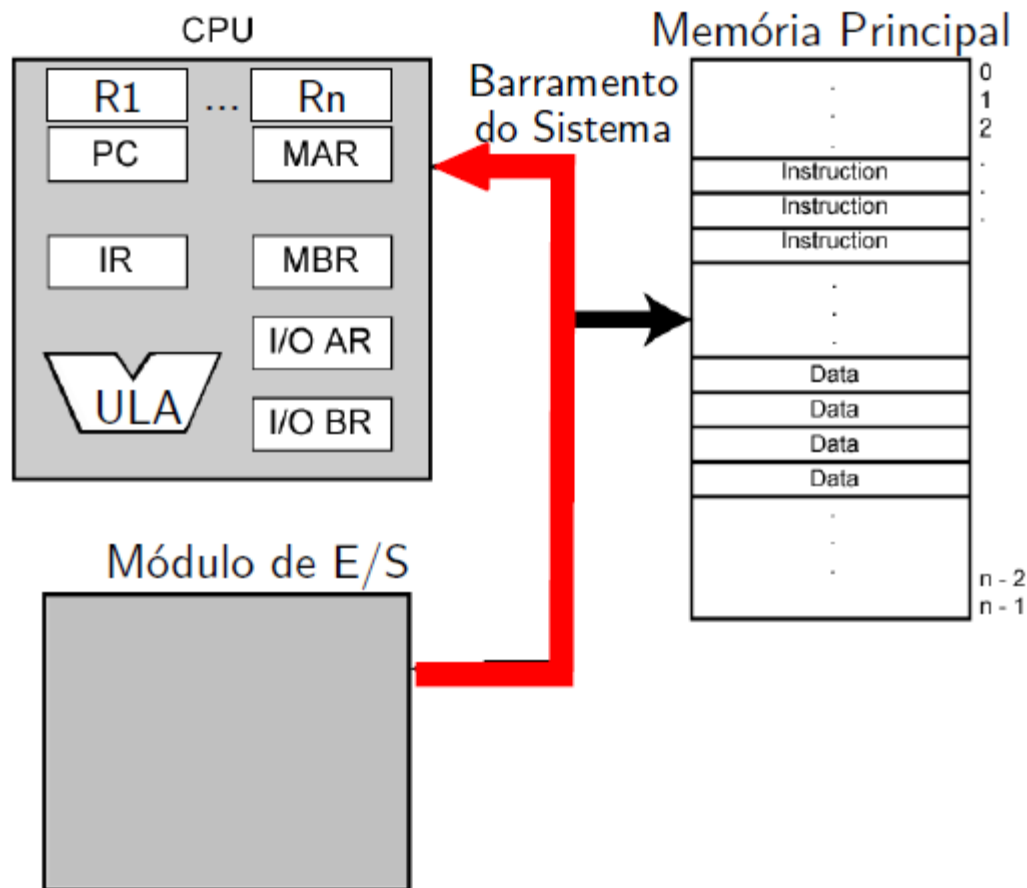
Tipos de Transferência

- Processador para memória principal



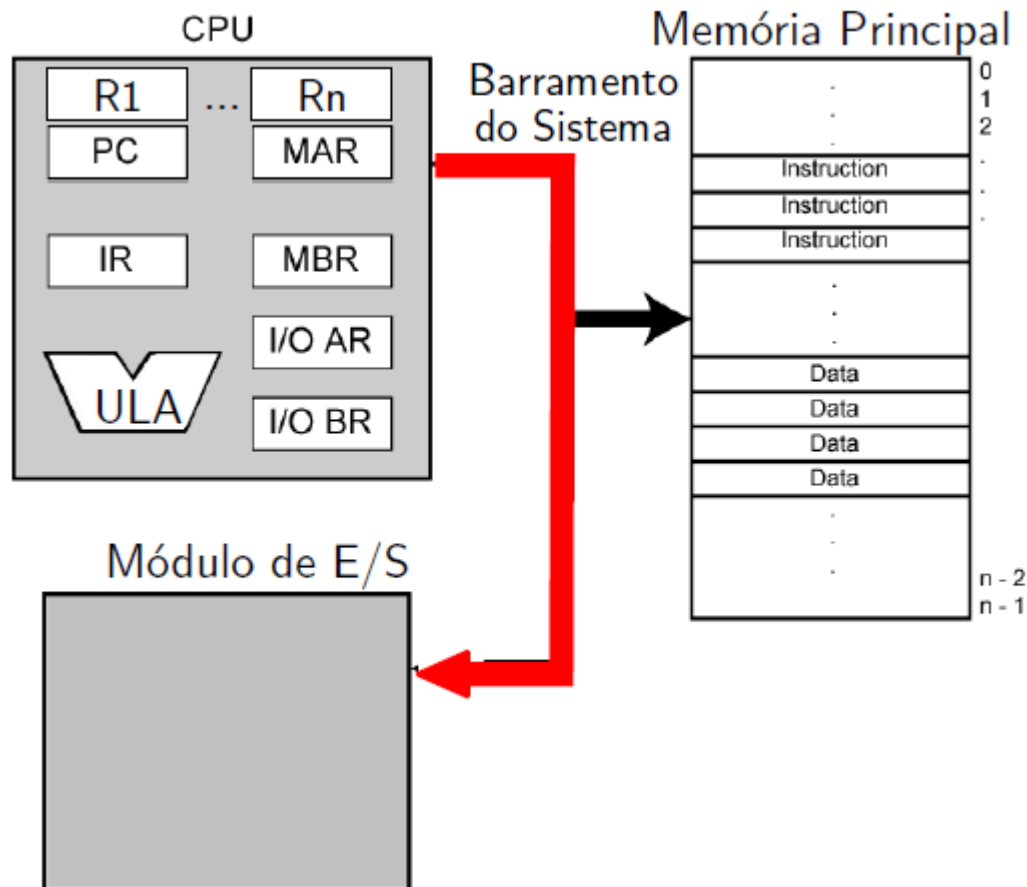
Tipos de Transferência

- Subsistema de E/S para processador



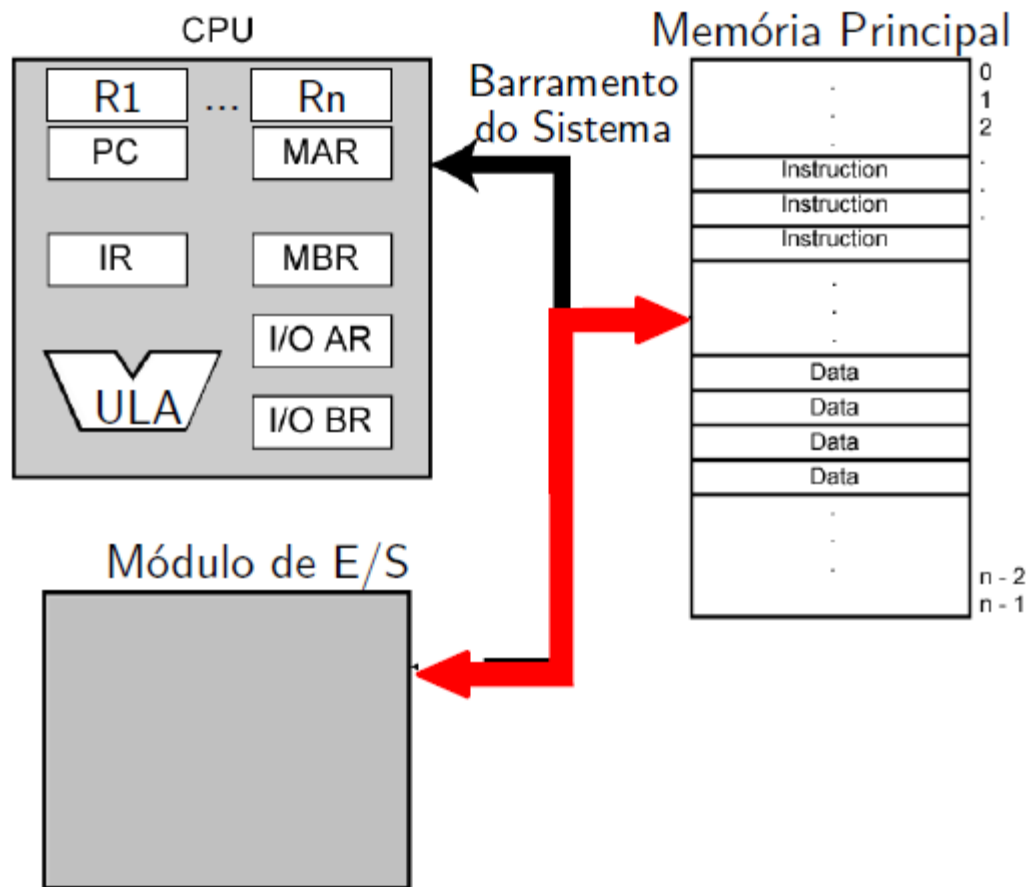
Tipos de Transferência

- Processador para subsistema de E/S



Tipos de Transferência

- Entre memória e subsistema de E/S (DMA) - Opcional



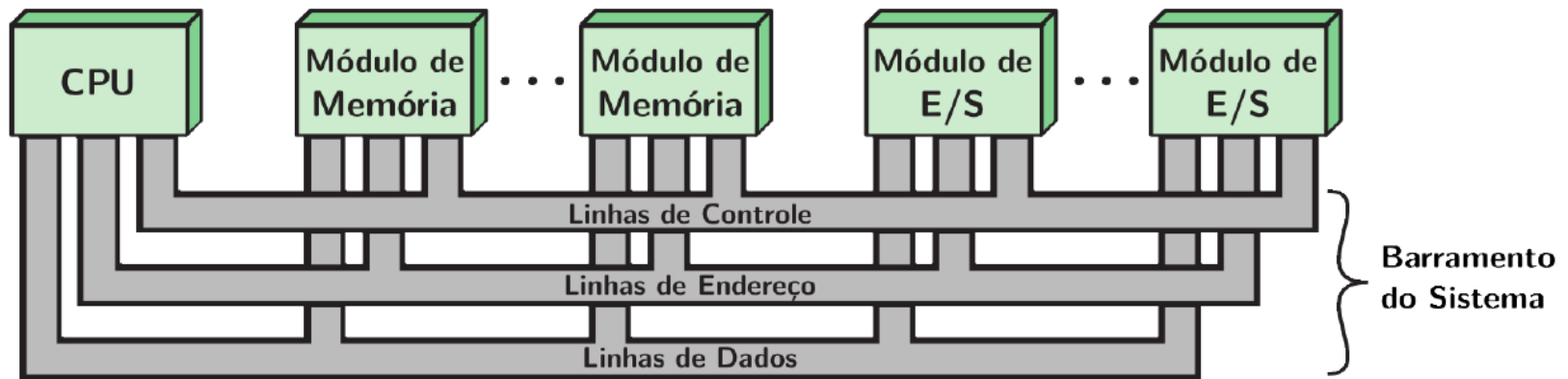
Barramento (Bus)

- Há diversas maneiras de se projetar um sistema de interconexão.
- Mais comuns:
 - Barramento separado em linhas de dados, endereço e controle.
 - Barramento compartilhado (multiplexado) para vários tipos de informação: Dados e endereço, por exemplo.

O que é Barramento?

- Meio de comunicação entre dois ou mais dispositivos.
- Sinais colocados em um barramento são recebidos por todos os dispositivos (*broadcast*).
- Apenas um módulo pode escrever em um barramento em um dado instante!
- Geralmente é agrupado:
 - Vários canais de comunicação em um só barramento (barramento paralelo).
 - Barramento de dados de 32 bits é um conjunto de 32 canais de 1 bit.
- Nos modelos não se representam as conexões para a fonte de tensão.

Diagrama de Interconexão



- No diagrama acima o barramento está dividido em:
 - Barramento de Dados;
 - Barramento de Endereço;
 - Barramento de Controle.

Barramento de Dados

- Transmite dados (obviamente!).
 - Observe que não há distinção entre “dado” e “instrução” neste nível (estamos fora da CPU);
- Largura do barramento é um fator determinante no desempenho (via de regra, quanto mais largo, melhor).
- Larguras de barramento de dados comuns:
 - 8 bits (Intel 8088);
 - 16 bits (Intel 8086);
 - 32 bits (Intel Pentium);
 - 64 bits (Intel Xeon – Core2Duo/Quad – Corei3/5/7).

Barramento de Endereço

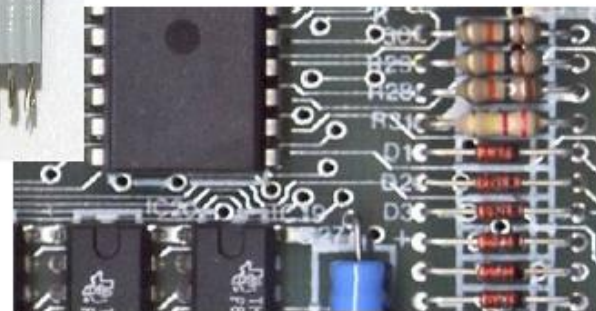
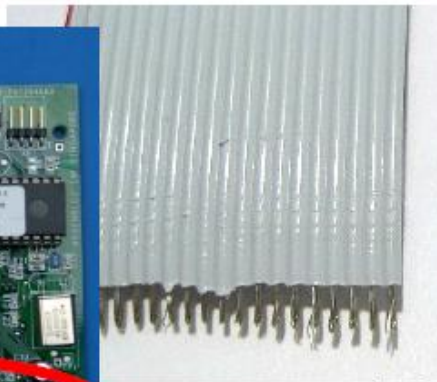
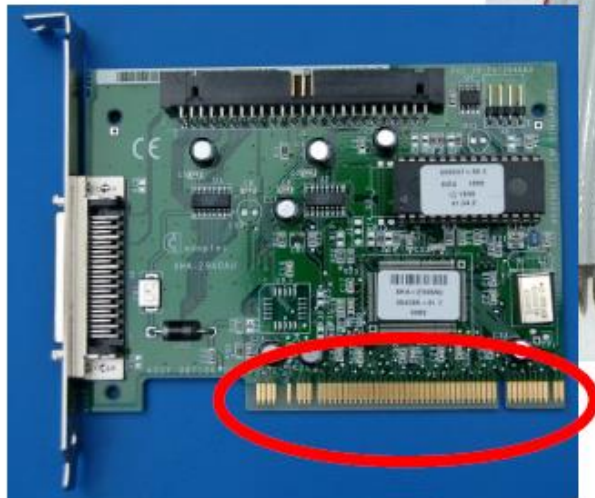
- Usado para identificar a fonte e o destino dos dados que trafegam no barramento de dados.
- Ex.: CPU precisa ler uma instrução (dado) de uma determinada posição na memória.
- Largura do barramento de endereço determina a capacidade de memória máxima do sistema.
- Ex.: Z-80 tem um barramento de endereço de 16 bits.
Memória máxima: $2^{16} = 65.536$ posições de memória.

Barramento de Controle

- Controle e temporização da informação:
 - Sinais de leitura e escrita para a memória principal ou para dispositivos de E/S;
 - Requisições de interrupção;
 - Sinais de controle gerais, tais como:
 - Requisição de uso do barramento por um módulo;
 - Término de uso do barramento;
 - Inicialização (reset) de dispositivos, etc.

Aspectos físicos dos barramentos

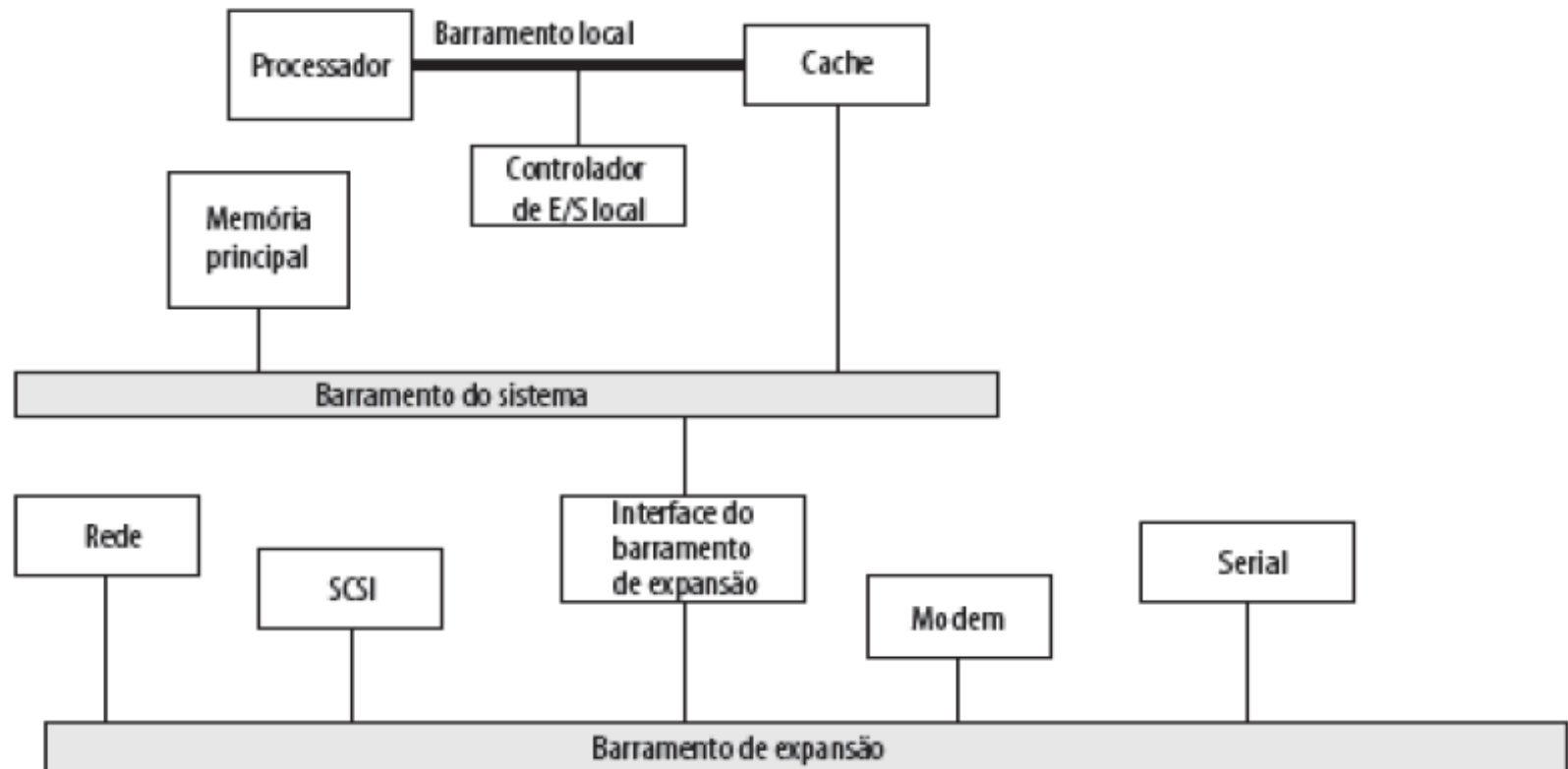
- Conectores de borda de placa de expansão (Ex.: PCI);
- Cabos paralelos (*ribbon cable*);
- Linhas paralelas em placas de circuito impresso;



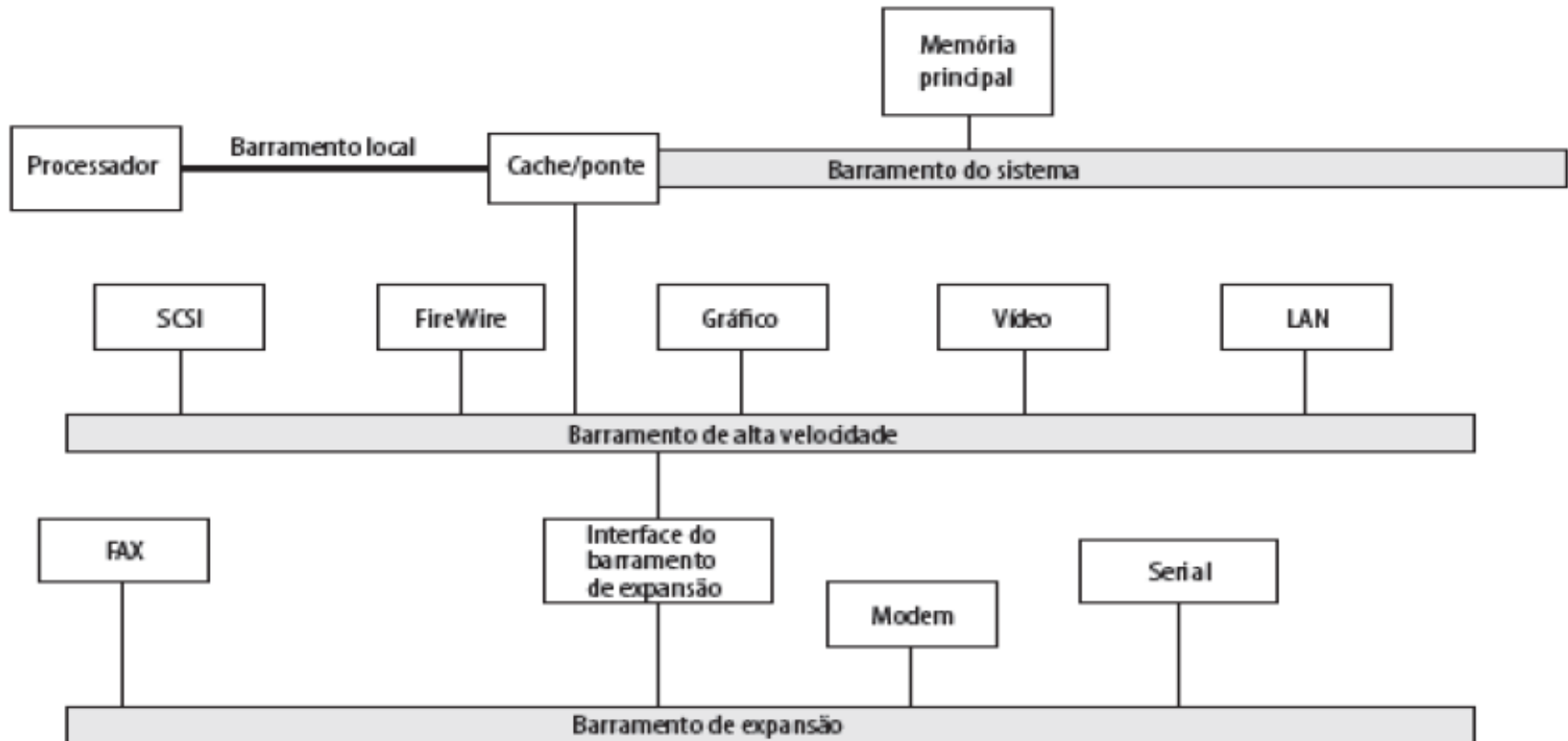
Barramento Único

- Apesar de ser mais simples de implementar, há desvantagens de se usar um único barramento de sistema.
 - Condutores longos: Tempo maior de propagação;
 - Muitos dispositivos ligados no mesmo meio de transmissão significa que a coordenação do uso do barramento pode afetar o desempenho geral do sistema;
 - E se houver dispositivos com diferentes velocidades de transmissão ligados ao barramento?
 - Taxa de transferência total de dados é limitada à taxa de transferência no barramento do sistema;
- Solução: Usar múltiplos barramentos!

Estrutura de barramento tradicional



Arquitetura de alto desempenho



Classificação de Barramentos

- **Dedicado**
 - Linhas separadas para dados e endereços;
- **Multiplexado**
 - Linhas compartilhadas;
 - Necessita de mais uma linha de controle, para indicar se há dados ou endereços trafegando no barramento naquele instante;
 - Vantagem: Necessita de menos condutores;
 - Desvantagens:
 - Controle mais complexo;
 - Pode diminuir o desempenho (pois compartilha um só canal para endereços e dados).

Arbitragem do Barramento

- Apenas um módulo pode controlar o barramento por vez (apesar de mais de um módulo poder ler o que está sendo transmitido).
- O que acontece se mais de um módulo solicitar o controle do barramento?
- Claramente, precisamos estabelecer regras para ceder o controle do barramento: **Arbitragem do barramento**.
- As regras definem mestre (que controla) e escravo(s) (que é/são controlado/os).
- A arbitragem pode ser centralizada ou distribuída.

Tipos de Arbitragem

- **Centralizada**
 - Somente um módulo controla o acesso ao barramento.
 - Controlador de barramento ou árbitro decide quem poderá controlar o barramento.
 - Pode ser parte da CPU, ou um dispositivo dedicado a essa tarefa.
- **Distribuída**
 - Cada módulo pode decidir controlar o barramento.
 - Necessária lógica de controle em todos os módulos.

Gargalo de von Neumann

- Refere-se ao tráfego nos barramentos do computador:
 - Vai endereço da instrução, volta instrução;
 - Vão endereços dos operandos;
 - Vão e voltam operandos.
- Para eliminar gargalo: Diminuir tráfego de informações.
 - Manter informações na CPU → Inclusão de registradores;
 - Diminuir tamanho (em bits) das informações transferidas.

Bibliografia

- Sttallings, W. **Arquitetura e organização de computadores: projeto para o desempenho**. 8. ed. Prentice Hall, 2009.